$Multi-level\ Linked\ List$

$-Print\ List(Main\ List\ and\ Child)$

$-Time\ Complexity\ Calculation\ using\ Recursive\ Tree\ Method$

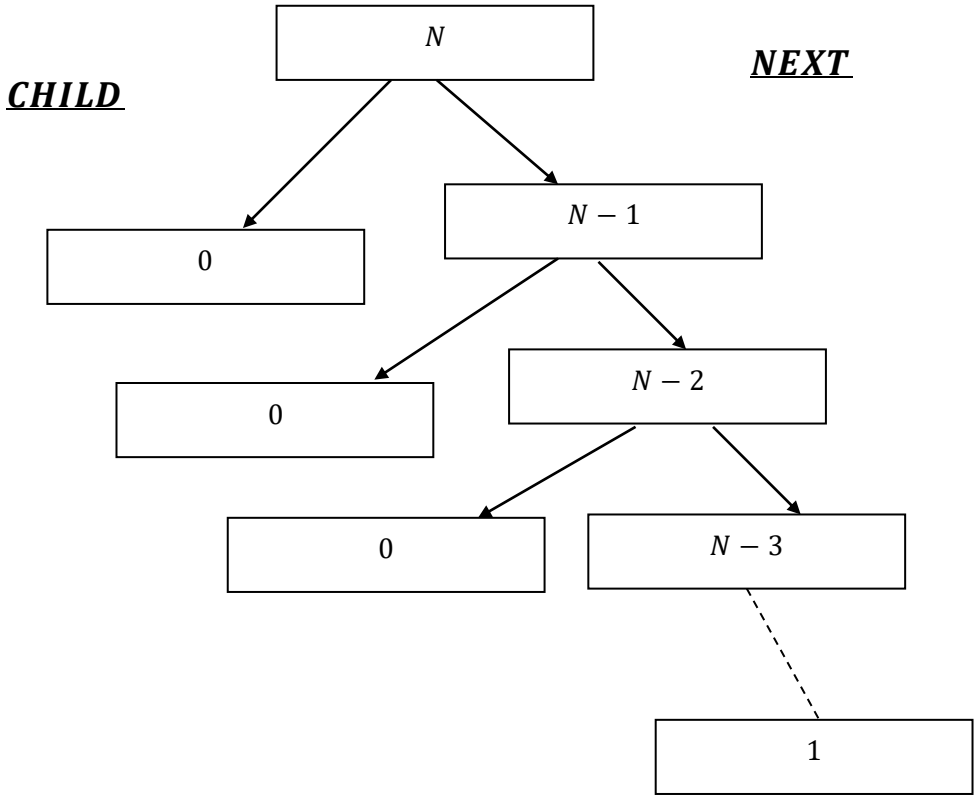
Program:

```
void printList(Node *node, int level = 0)
{
    while (node != nullptr)
    {
        for (int i = 0; i < level; i++)
            cout << "    ";
        cout << "|-- " << node->data << endl;

        if (node->child != nullptr)
        {
            printList(node->child, level + 1);
        }
        node = node->next;
    }
}
….
       case 9:
            cout << "Current List Structure: " << endl;
            printList(head);
            break;
```

$Imbalanced\ (Flat)$

$$T(n) = \begin{cases} 1 & for\ n = 0 \\ T(N-1) + c & for\ n > 0 \end{cases}$$



Here ,while loop runs but for loop never runs.

$\boldsymbol{Step/\ Depth(k)}$	$\boldsymbol{Nodes\ Remaining\ (i)}$	$\boldsymbol{Node's\ Level\ (d_i)}$	$\boldsymbol{Time\ in\ for\ loop\ c_1 \times d_i}$	$\boldsymbol{Constant\ Work\ (c_2)}$	$\boldsymbol{Total\ Work\ (c_1 \times d_i + c_2)}$	$\boldsymbol{Current\ Equation}$
$\mathbf{1}$	$\boldsymbol{N}$	$\mathbf{0}$	$\boldsymbol{c_1 \times 0 = 0}$	$\boldsymbol{c_2}$	$\boldsymbol{0 + c_2 = c_2}$	$\boldsymbol{T(N) = T(N-1) + c}$
$\mathbf{2}$	$\boldsymbol{N-1}$	$\mathbf{0}$	$\boldsymbol{c_1 \times 0 = 0}$	$\boldsymbol{c_2}$	$\boldsymbol{0 + c_2 = c_2}$	$\boldsymbol{T(N-1) = T(N-2) + c}$
$\mathbf{3}$	$\boldsymbol{N-2}$	$\mathbf{0}$	$\boldsymbol{c_1 \times 0 = 0}$	$\boldsymbol{c_2}$	$\boldsymbol{0 + c_2 = c_2}$	$\boldsymbol{T(N-2) = T(N-3) + c}$
….	….	….	….	….	….	….
$\boldsymbol{N}$	$\mathbf{1}$	$\mathbf{0}$	$\boldsymbol{c_1 \times 0 = 0}$	$\boldsymbol{c_2}$	$\boldsymbol{0 + c_2 = c_2}$	$\boldsymbol{T(1) = T(0) + c}$
$\boldsymbol{Base\ Case}$	$\mathbf{0}$	$\boldsymbol{N/A}$	$\boldsymbol{N/A}$	$\boldsymbol{c_2}$	$\boldsymbol{0 + c_2 = c_2}$	$\boldsymbol{T(0) = c}$

In general:

$\boldsymbol{Level}$ (k)	$\boldsymbol{No.of\ problems}$	$\boldsymbol{Problem\ Size}$	$\boldsymbol{Work\ /Cost\ Per\ Problem}$	$\boldsymbol{Work\ done = Work\ per\ Problem \times No.of\ Problems}$
$\boldsymbol{0}$	$\boldsymbol{1}$	$\boldsymbol{n-0=n}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
$\boldsymbol{1}$	$\boldsymbol{1}$	$\boldsymbol{n-1}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
$\boldsymbol{2}$	$\boldsymbol{1}$	$\boldsymbol{n-2}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
. .	. .	. .	. .	. .
$\boldsymbol{n-1}$	$\boldsymbol{1}$	$\boldsymbol{n-(n-1)=1}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
$\boldsymbol{n}$ $(\boldsymbol{Base\ Case})$	$\boldsymbol{1}$	$\boldsymbol{n-n=0}$ $(\boldsymbol{Base\ Case})$	$\boldsymbol{T(0)}$	$\boldsymbol{1\times T(0)=T(0)}$

$\boldsymbol{Why\ level\ `n`\ is\ Base\ Case?}$

$\boldsymbol{Ans: for, level\ `k`:, Base\ Case = T(0)\ \therefore n-k=0}$

$\boldsymbol{\Rightarrow k=n.}$

$\boldsymbol{Total\ Cost = C+C+C+\cdots(N)times+T(0).}$

$\boldsymbol{=C(N)+T(0)}$

$\boldsymbol{We\ know, T(0)is\ 1, we\ can\ go\ with\ 1, but\ let\ instead\ lets\ write\ T(0)}$

$\boldsymbol{=C_0, representing\ constant.}$

$$Total\ Cost = C(N) + C_0$$

$As, C\ and\ C_0\ both\ are\ constant\ Applying\ Big\ O\ , we\ get$:

$$= O(N)$$

$Alternate, in\ microscopic\ level$:

$$T(n) = \begin{cases} 1 & for\ n = 0 \\ T(N-1) + c & for\ n > 0 \end{cases}$$

$This\ is\ a\ generalised\ version$:

$$T(N-1) + c \qquad for\ n > 0$$

$But\ actual\ equation$:

$The\ printList\ function\ contains\ a\ for\ loop\ that\ runs\ based\ on\ the\ current\ level$:

```
        for (int i = 0; i < level; i++)
            cout << "    ";
```

$This\ means\ as\ the\ recursive\ tree\ gets\ deeper, the\ work\ per\ node\ increases.$

$Let's\ define\ the\ new\ cost$:

- $Let\ c_2\ be\ the\ constant\ work\ at\ the\ node\ (checking\ nullptr, printing\ node \rightarrow data, etc.)$.
- $Let\ c_1\ be\ the\ cost\ of\ printing\ a\ single\ space\ in\ the\ loop.$
- $Because\ level\ corresponds\ to\ the\ tree's\ depth\ k, the\ loop\ runs\ k\ times.$

$$\textbf{\textit{Therefore, Cost Per Problem at level `k` }} = c_1 \cdot k + c_2.$$

Recursion Step	***No. of problems***	***Problem Size***	***Node's Level*** (k)	***Work /Cost Per Problem*** $(c_1 \times k + c_2)$	***Work done at Level***
$\mathbf{0}$	$\mathbf{1}$	$n - 0 = n$	$\mathbf{0}$	$c_1(0) + c_2$	c_2
$\mathbf{1}$	$\mathbf{1}$	$n - 1$	$\mathbf{0}$	$c_1(0) + c_2$	c_2
$\mathbf{2}$	$\mathbf{1}$	$n - 2$	$\mathbf{0}$	$c_1(0) + c_2$	c_2
. .	. .	. .	. .	. .	. .
$n - 1$	$\mathbf{1}$	$n - (n - 1) = 1$	$\mathbf{0}$	$c_1(0) + c_2$	c_2
n (***Base Case***)	$\mathbf{1}$	$n - n = 0$ (***Base Case***)	N/A	$T(0) = 1$	$1 \times T(0) = T(0)$

$$\textbf{\textit{Total Cost}} = c_2 + c_2 + c_2 + \cdots (n-1)\textbf{\textit{times}} + T(0)$$

$$\textbf{\textit{Total Cost}} = c_2(1 + 1 + 1 + \cdots (n-1)\textbf{\textit{times}}) + T(0)$$

$$\textbf{\textit{Total Cost}} = c_2\left(\sum_{i=1}^{n-1} 1_i\right) + T(0)$$

Substituting $T(0) = 1$ ***, we get***:

$$Total\ Cost = c_2(1 \times (n-1)) + 1$$

$$Total\ Cost = c_2(n-1) + 1$$

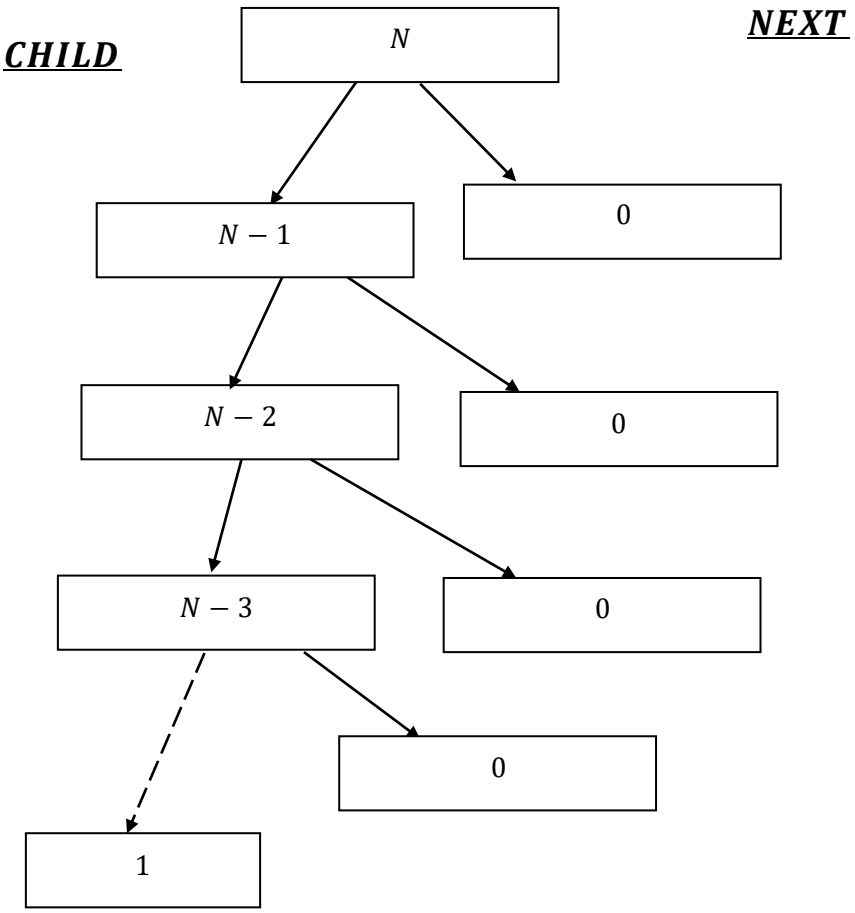
$$Total\ Cost = c_2 n - c_2 + 1$$

Applying Big O , we get:

$$Total\ Cost = O(n)$$

$\boldsymbol{Imbalanced\ (Deep)}$

$$\boldsymbol{T(n)} = \begin{cases} \mathbf{1} & \boldsymbol{for\ n = 0} \\ \boldsymbol{T(N-1) + c\ \times N} & \boldsymbol{for\ n > 0} \end{cases}$$



Here ,while loop runs , for loop runs.

Step/ Depth(k)	***Nodes Remaining (i)***	***Node's Level (d_i)***	***Time in for loop $c_1 \times d_i$***	***Constant Work (c_2)***	***Total Work ($c_1 \times d_i + c_2$)***	***Current Equation***
1	N	**0**	$c_1 \times 0 = 0$	c_2	$0 + c_2$	$T(N) = T(N-1) + c_2$
2	$N-1$	**1**	$c_1 \times 1 = c_1$	c_2	$c_1 + c_2$	$T(N-1) = T(N-2) + (c_1 + c_2)$
3	$N-2$	**2**	$c_1 \times 2 = 2c_1$	c_2	$2c_1 + c_2$	$T(N-2) = T(N-3) + (2c_1 + c_2)$
……	……	……	……	……	……	……
N	**1**	$N-1$	$c_1 \times (N-1)$	c_2	$c_1 \times (N-1) + c_2$	$T(1) = T(0) + (c_1 \times (N-1) + c_2)$
Base Case	**0**	N/A	N/A	c_2	$0 + c_2 = c_2$	$T(0) = c$

In general :

The printList function contains a for loop that runs based on the current level:

```
        for (int i = 0; i < level; i++)
            cout << "    ";
```

This means as the recursive tree gets deeper, the work per node increases.

***Let's define the new cost*:**

- ***Let c_2 be the constant work at the node* (*checking nullptr, printing node → data, etc.*).**
- ***Let c_1 be the cost of printing a single space in the loop*.**
- ***Because level corresponds to the tree's depth k, the loop runs k times*.**

$$\textbf{Therefore, Cost Per Problem at level `k` } = c_1 \cdot k + c_2.$$

Level (k)	***No. of problems***	***Problem Size***	***Work /Cost Per Problem***	***Work done = Work per Problem × No. of Problems***
0	**1**	$n - 0 = n$	$C \times N$	$1 \times (C \cdot N) = CN$
1	**1**	$n - 1$	$C \times (N - 1)$	$1 \times (C \cdot N - 1) = C(N - 1)$
2	**1**	$n - 2$	$C \times (N - 2)$	$1 \times (C \cdot N - 2) = C(N - 2)$
. .	. .	. .	. .	. .
$n - 1$	**1**	$n - (n - 1) = 1$	$C \times 1$	$1 \times C = C$
n (***Base Case***)	**1**	$n - n = 0$ (***Base Case***)	$T(0)$	$1 \times T(0) = T(0)$

$$\boldsymbol{TotalCost = cN + c(N-1) + c(N-2) + \cdots + c + T(0)}$$

$$\boldsymbol{TotalCost = c \cdot (N + (N-1) + (N-2) + \cdots + 1) + T(0)}$$

$$\boldsymbol{TotalCost = c \cdot \left[\frac{N(N+1)}{2}\right] + T(0)}$$

Substitute your specific base case $\boldsymbol{(T(0) = 1)}$:

$$\boldsymbol{TotalCost = c \cdot \left[\frac{N^2 + N}{2}\right] + 1}$$

Apply $\boldsymbol{Big - O\ Notation}$: ***By removing the constant multipliers*** $\boldsymbol{(like\ c\ and\ the\ division\ by\ 2)}$, ***removing the non*** $\boldsymbol{- dominant\ linear\ +\ N\ term}$, ***and dropping the*** $\boldsymbol{+\ 1}$ ***constant at the end, we are left purely with the fastest*** $\boldsymbol{- growing\ term}$:

$$\boldsymbol{= O(N^2)}$$

Alternate, in microscopic level:

$$T(n) = \begin{cases} 1 & for\ n = 0 \\ T(N-1) + c \times N & for\ n > 0 \end{cases}$$

This is a generalised version :

$$T(N-1) + c \times N \qquad for\ n > 0$$

But actual equation:

The printList function contains a for loop that runs based on the current level:

```
        for (int i = 0; i < level; i++)
            cout << "    ";
```

This means as the recursive tree gets deeper, the work per node increases.

Let's define the new cost:

- *Let c_2 be the constant work at the node (checking nullptr, printing node → data, etc.).*
- *Let c_1 be the cost of printing a single space in the loop.*
- *Because level corresponds to the tree's depth k, the loop runs k times.*

Therefore, Cost Per Problem at level `k` = $c_1 \cdot k + c_2$.

$$T(N-1) + c_1 \times k + c_2 \qquad for\ n > 0$$

Recursion Step	No. of problems	Problem Size	Node's Level (k)	Work /Cost Per Problem ($c_1 \times k + c_2$)	Work done at Level
0	1	$n - 0 = n$	0	$c_1(0) + c_2$	c_2
1	1	$n - 1$	1	$c_1(1) + c_2$	$c_1 + c_2$
2	1	$n - 2$	2	$c_1(2) + c_2$	$2c_1 + c_2$
. .	. .	. .	. .	. .	. .
$n - 1$	1	$n - (n-1) = 1$	$n - 1$	$c_1(n-1) + c_2$	$c_1(n-1) + c_2$
n (Base Case)	1	$n - n = 0$ (Base Case)	N/A	$T(0) = 1$	$1 \times T(0) = T(0)$

$$TotalCost = c_2 + (c_1 + c_2) + (2c_1 + c_2) + (3c_1 + c_2) + \cdots + (c_1(n-1) + c_2) + T(0).$$

$$= c_2 + (c_1 + c_2) + (2c_1 + c_2) + (3c_1 + c_2) + \cdots + (c_1(n-1) + c_2) + T(0).$$

$$= ((c_1 + 2c_1 + 3c_1 + \cdots + c_1(n-1)) + (c_2 + c_2 + \cdots n\ times) + T(0))$$

$$= ((c_1 + 2c_1 + 3c_1 + \cdots + c_1(n-1)) + (c_2 + c_2 + \cdots n\ times) + T(0))$$

$$= ((c_1 + 2c_1 + 3c_1 + \cdots + c_1(n-1)) + (c_2 + c_2 + \cdots n\ times) + T(0))$$

$$= \left(c_1\left(1 + 2 + 3 + \cdots + (N-1)\right) + N \times c_2 + T(0)\right)$$

$$= \left(c_1\left(\sum_{i=1}^{n-1} i\right) + N \times c_2 + T(0)\right)$$

$$= \left(c_1\left(\frac{n(n-1)}{2}\right) + N \times c_2 + T(0)\right)$$

$$= \left(c_1\left(\frac{n^2 - n}{2}\right) + N \times c_2 + T(0)\right)$$

Substituting $T(0) = 1$, ***we get***:

$$= \left(c_1\left(\frac{n^2 - n}{2}\right) + N \times c_2 + 1\right)$$

Applying Big O, we get:

$$= O(n^2)$$

$$\underline{\boldsymbol{Best\ Structure\ [For\ Balanced\ Tree]}}$$

$$2T\left(\frac{N-1}{2}\right)+C\ ,Base\ Case: T(0)=1.$$

Work Per Problem or Cost Per Problem :

```
void printList(Node *node, int level = 0)
{
    while (node != nullptr)
    {
        for (int i = 0; i < level; i++)
            cout << "    ";
        cout << "|-- " << node->data << endl;

        if (node->child != nullptr)
        {
            printList(node->child, level + 1);
        }
        node = node->next;
    }
}
….
       case 9:
            cout << "Current List Structure: " << endl;
            printList(head);
            break;
```

$\boldsymbol{Level\ (k)}$	$\boldsymbol{No.of\ problems}$	$\boldsymbol{Problem\ Size}$	$\boldsymbol{Work\ /Cost\ Per\ Problem}$	$\boldsymbol{Work\ done = Work\ per\ Problem \times No.of\ Problems}$
$\mathbf{0}$	$\mathbf{1}$	n	C	$1 \times C = C$
$\mathbf{1}$	2	$\frac{n-1}{2}$	C	$2 \times C = 2C$
$\mathbf{2}$	4	$\frac{n-3}{4}$	C	$4 \times C = 4C$
$\mathbf{3}$	$\mathbf{8}$	$\frac{n-7}{8}$	C	$8 \times C = 8C$
. .	. .	. .	. .	. .
k	2^k	$\frac{N-(2^k-1)}{2^k}$	C	$2^k \times C = 2^k C$
$(Base\ Case)$	$N+1$	$\frac{N-(2^{\log_2(N+1)}-1)}{2^{\log_2(N+1)}} = 0$ $(Base\ Case)$	$T(0)$	$(N+1) \times T(0)$

$Q) Why\ `N+1`\ Base\ Case?$

$The\ "Pointer\ Math"\ Proof$

$Imagine\ of\ building\ a\ perfectly\ balanced\ multi-level\ linked\ list$

$with\ exactly\ N\ real\ nodes.$

1. ***Total Pointers***: ***Every single node in your structure contains exactly two pointers*** (*child* ***and*** *next*). ***Therefore, in a list of N nodes, there are exactly 2N total pointers physically sitting in memory***.

2. ***Connecting the Nodes***: ***To connect N nodes together into a single tree, we have to use exactly*** $N - 1$ ***pointers***.
   (***Think about it***: ***to connect*** 2 ***nodes, it need*** 1 ***pointer***. ***To connect*** 3 ***nodes, need*** 2 ***pointers***).

3. ***The Leftovers***: ***If we have*** $2N$ ***total pointers, and we used*** $N - 1$ ***of them to wire the structure together, how many pointers are left pointing to absolutely nothing***?

Let's do the math:

$$2N - (N - 1)$$
$$= 2N - N + 1$$
$$= N + 1$$

What does this mean for your code?

PrintList ***code is smarter***; ***it looks before it leaps***. ***It explicitly checks if the pointers are valid before trying to process them***. ***Those*** $N + 1$ ***leftover pointers pointing to*** *nullptr* ***are the exact moments of code's specific conditions evaluate to false***.

In printList function, evaluation of those N + 1 nullptrs is split across two specific lines of code:

1. *The horizontal dead end*: *The condition while (node != nullptr) evaluating to false when you reach the end of a next list*.
2. *The vertical dead end*: *The condition if (node → child != nullptr) evaluating to false when a node has no deeper child levels*.

Even though it is written differently, the CPU still has to evaluate a nullptr condition exactly N + 1 times across the entire data structure.

Where are the nullptrs hiding?

Every single node created in this balanced structure has exactly 2 pointers (child and next). Depending on where the node is in the tree, those pointers might point to real data, or they might point to nothing.

Here is exactly how printList code encounters them, mapped to the three types of nodes:

1. *The* Leaf *Nodes (The very bottom)*: *If a node is at the very bottom of the structure, it has no children and no next nodes*.

   Both of its pointers are nullptr.

Both of its pointers are nullptr.

- *if* (*node* → *child* != *nullptr*).*This evaluates to false* (1*st nullptr found*).
- *When code executes node* = *node* → *next*;, *which sets node to nullptr.*
- *The loop circles back up and checks while* (*node* != *nullptr*). *This evaluates to false and breaks the loop* (2*nd nullptr found*).
- *Total contribution to the base case*: 2.

2. *The* Half-Full *Nodes* (*Near the bottom*):

If the tree isn't perfectly balanced, might have nodes that only have one valid pointer.

- *Scenario A* (*Has a child, but no next node*): *The child check is true and processes the child branch. Then, node* = *node* → *next sets the current node to nullptr. The loop circles back and the while* (*node* != *nullptr*)*check evaluates to false.* (1 *nullptr found*).
- *Scenario B* (*Has a next node, but no child*): *The if* (*node* → *child* != *nullptr*)*check evaluates to false* (1 *nullptr found*). *The node* = *node* → *next points to a real node, so the while check remains true.*
- *Total contribution to the base case*: 1.

3. ***The*** Full***Nodes*** (***The middle***): ***If a node has both a child and a next, the function follows both to real nodes.***

- ***Code checks if*** (***node*** –> ***child*** ! = ***nullptr***).***It evaluates to true.***
- ***Code executes node*** = ***node*** → ***next***; .***The new node is real, so while*** (***node*** ! = ***nullptr***) ***evaluates to true.***
- ***Because neither check failed, no nullptrs were found at this exact step.***

- ***Total contribution to the base case***: **0.**

Combining these **1, 2** ***cases*** = ***N*** + **1**. ***i. e. How many times bases cases are hit***?

N + **1** ***times***,

Just to summarize that golden rule of binary trees:

- ***We have N real nodes.***
- ***They contain 2N total pointers.***
- ***We use N*** − **1** ***of those pointers to connect the tree together.***
- ***We are left with exactly N*** + **1** ***nullptr pointers.***

No matter how the N real nodes are arranged, they physically contain 2N total pointers. N − **1** ***of them used to connect the tree,***

meaning the remaining exactly N + **1** ***pointers are nullptr.***

Because while loop and if statement must check every single pointer to know when to stop traversing, printlist code is mathematically guaranteed to evaluate a false condition against nullptr exactly N + **1** ***times, regardless of the tree's exact shape.***

This is exactly where $(N+1) \times T(0)$ ***base case math comes from***!

Now we have to derive level `k` first:

$$\textbf{Problem size} = \frac{N - \left(2^k - 1\right)}{2^k}$$

$$\textbf{And we know}, T(0) \textbf{is our base case}\ .$$

The tree stops growing $(\textit{it hits the base case})$ ***when the problem size reaches*** 0.

So, to find the final level k, ***we set our pattern equal to*** 0 ***and solve for*** k:

$$\frac{N - \left(2^k - 1\right)}{2^k} = 0$$

Multiplying 2^k ***on both the sides***:

$$\Longrightarrow N - \left(2^k - 1\right) = 0$$

Moving the $2^k - 1$ ***to the other side, we get***:

$$\Longrightarrow N = 2^k - 1$$

Add 1 ***to both sides to isolate the exponential term***:

$$\Longrightarrow N + 1 = 2^k$$

Take the base -2 ***logarithm of both sides***.

Because the base of our exponent is $2 (\textit{from}\ 2^k)$, ***we use the base*** -2 ***logarithm*** $(\log_2)$ ***to target it***.

$$\Longrightarrow \log_2(N+1) = \log_2 2^k$$

$$\Longrightarrow \log_2(N+1) = \mathrm{k} \times \log_2 2$$

$$\Longrightarrow \log_2(N+1) = k \ [\log_a a = 1]$$

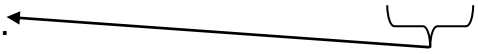
$$\therefore k = \log_2(N+1)$$

Hence full table is:

Level (k)	***No. of problems***	***Problem Size***	***Work /Cost Per Problem (Print and free memory)***	***Work done = Work per Problem × No. of Problems***
0	1	n	C	$1 \times C = C$
1	2	$\frac{n-1}{2}$	C	$2 \times C = 2C$
2	4	$\frac{n-3}{4}$	C	$4 \times C = 4C$
3	8	$\frac{n-7}{8}$	C	$8 \times C = 8C$
. .	. .	. .	. .	. .
k	2^k	$\frac{N-(2^k-1)}{2^k}$	C	$2^k \times C = 2^k C$
. . .	. . .	. . .	. . .	. . .
$\log_2(N+1)$ $(Base\ Case)$	$N+1$	$\frac{N-(2^{\log_2(N+1)}-1)}{2^{\log_2(N+1)}} = 0$ $(Base\ Case)$	$T(0)$	$(N+1) \times T(0)$

$Also\ , no.of\ problems : 2^k, when\ k = \log_2(N+1) \Longrightarrow 2^{\log_2(N+1)} \Longrightarrow N+1$

$We\ can\ also\ take$ from base level : $\log_2(N+1), Number\ of\ Problem\ is : N+1$.



$$As, 2^k = N+1.$$

$$\begin{bmatrix} From\ Mathematical\ Terms\ apart\ from \\ Pointer\ terms\ explanation. \end{bmatrix}$$

$Now\ lets\ calculate$:

$Now\ , we\ can\ see\ growth : C + 2C + 4C + \cdots + 2^{k-1}C + (N+1) \times T(0).$

$Note\ , 2^k = N+1 (Already\ proved\ above)$

$\therefore The\ series\ is\ actually,$

$\Rightarrow C + 2C + 4C + \cdots + 2^{k-1}C + 2^k \times T(0)\ [Where\ 2^k = N+1]$

$$\begin{bmatrix} We\ know\ T(0)\ is\ Constant \\ But\ already\ we\ know\ its\ value. \end{bmatrix}$$

$Hence\ , C + 2C + 4C + \cdots + 2^{k-1}C + (N+1) \times T(0).$

$\Longrightarrow C\left(1 + 2 + 4 + \cdots + 2^{k-1}\right) + (N+1) \times T(0).$

$Applying\ geometric\ progression (series)$:

$$S(n) = ar^0 + ar^1 + ar^2 + \cdots + ar^n = a\left(\frac{r^{n+1}-1}{r-1}\right), where$$

$S(n)$ *is sum of first n terms ,a is coefficient and* r *is common ratio between the consecutive terms.*

We get:

$$\Longrightarrow C(1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2 + \cdots + 1 \times 2^{k-1}) + (N+1) \times T(0).$$

$$\Rightarrow C\left(1 \times \left(\frac{2^{k-1+1}-1}{2-1}\right)\right) + (N+1) \times T(0).$$

or we can directly apply geometric series ,for $[a \times r^{n-1}]$

$$= a\left(\frac{r^n-1}{r-1}\right)$$

$$\Rightarrow C\left(1 \times \left(\frac{2^k-1}{2-1}\right)\right) + (N+1) \times T(0).$$

$$\Rightarrow C\left(2^k-1\right) + (N+1) \times T(0).$$

We know , $k = \log_2(N+1)$ *, applying in the Equaltion we get*:

$$\Rightarrow C\left(2^{\log_2(N+1)}-1\right) + (N+1) \times T(0).$$

$$\Rightarrow C((N+1)-1) + (N+1) \times T(0). \left[Note, a^{log_a b} = b\right]$$

$$\Rightarrow CN + (N+1) \times T(0)$$

We know $T(0) = 1$ *, we can apply that here but instead we can write,*

$T(0) or\ t_0\ as\ C_0\ representing\ constant$:

$\Rightarrow CN + (N+1) \times C_0$

$Applying\ Big\ O\ , in\ the\ equation\ removing\ all\ the\ constants$:

$\Longrightarrow \mathbf{O}(CN + (N+1) \times C_0)$

$\Longrightarrow \mathbf{O}(N) + \mathbf{O}(N+1)$

$\Longrightarrow \mathbf{O}(N) + \mathbf{O}(N+1)$

$\Longrightarrow \mathbf{O}(N) + O(N)$

$\Longrightarrow \mathbf{O}(N+N)$

$\Longrightarrow \mathbf{O}(2N)$

$\Longrightarrow \mathbf{O}(N)$

Alternatively,

$Note:, if\ we\ take\ C\ and\ C_0\ as\ 1\ unit\ time\ , which\ will\ lead\ to\ the\ equation$:

$= N + N + 1$

$= 2N + 1$

$= O(N)[Alternatively]$

****

Alternate Calculation:

This is a generalized Version:

$$2T\left(\frac{N-1}{2}\right) + C\,,\, Base\ Case: T(0) = 1.$$

But printList function contains a for loop that runs based on the current level:

```
        for (int i = 0; i < level; i++)
            cout << "    ";
```

This means as the recursive tree gets deeper, the work per node increases.

Let's define the new cost:

- *Let c_2 be the constant work at the node (checking nullptr, printing node–> data, etc.).*

```
while (node != nullptr)
if (node->child != nullptr)
cout << "|-- " << node->data << endl;
node = node->next;
int i = 0 ; i < level
```

And call stak push on :

```
printList(node->child, level + 1);
```

With adding : Level + 1.

- ***Let c_1 be the cost of printing a single space in the loop.***

```
cout << "    ";
```

- ***Because level corresponds to the tree's depth k, the loop runs k times.***

Therefore, Cost Per Problem at level $k = c_1 \cdot k + c_2$.

$$2T\left(\frac{N-1}{2}\right) + c_1 \times k + c_2 \text{ , Base Case: } T(0) = 1.$$

Recursion Step	No. of problems	Problem Size	Node's Level (k)	Work /Cost Per Problem ($c_1 \times k + c_2$)	Work done at Level
0	1	n	0	$c_1(0) + c_2$	c_2
1	2	$\frac{n-1}{2}$	1	$c_1(1) + c_2$	$2 \times (c_1 + c_2) = 2c_1 + 2c_2$
2	4	$\frac{n-3}{4}$	2	$c_1(2) + c_2$	$4 \times (2c_1 + c_2) = 8c_1 + 4c_2$
3	8	$\frac{n-7}{8}$	3	$c_1(3) + c_2$	$8 \times (3c_1 + c_2) = 24c_1 + 8c_2$
. . .	. .	. .	. .	. .	. .
k	2^k	$\frac{N-(2^k-1)}{2^k}$	k	$c_1(k) + c_2$	$2^k \times (c_1 \cdot k + c_2)$
$\log_2(N+1)$ (Base Case)	$N+1$	$\frac{N-(2^{\log_2(N+1)}-1)}{2^{\log_2(N+1)}} = 0$ (Base Case)	N/A	$T(0) = 1$	$(N+1) \times T(0)$

$$\Rightarrow 2^0 \times (0 \times c_1 + c_2) + \left(2^1 \times (1 \times c_1 + c_2)\right) + (4 \times (2c_1 + c_2)) + (8 \times (3c_1 + c_2)) + \cdots$$

$$+2^k \times (c_1 \cdot k + c_2) + (N + 1) \times T(0)$$

$$\Longrightarrow (0 \cdot c_1 + 1 \cdot c_2) + (2c_1 + 2c_2) + (8c_1 + 4c_2) + (24c_1 + 8c_2) + \cdots +$$

$$\left(2^k \times kc_1 + 2^k c_2\right) + (N + 1) \times T(0)$$

$$T(N) =$$

$$c_1((0 \times 2^0) + (1 \times 2^1) + (2 \times 2^2) + (3 \times 2^3) + \cdots + \left(k \times 2^k\right) +$$

$$c_2((2^0) + (2^1) + (2^2) + (2^3) + \cdots + \left(2^k\right)) +$$

$$(N + 1) \times T(0)$$

$$T(N) = c_1\left(\sum_{i=0}^{k} i \times \sum_{j=0}^{k} 2^j\right) + c_2\left(\sum_{i=0}^{k} 2^i\right) + (N + 1) \times T(0)$$

$$T(N) = c_1\left(\sum_{i=0}^{k} i \times 2^i\right) + c_2\left(\sum_{i=0}^{k} 2^i\right) + (N + 1) \times T(0)$$

Let,

$$S_1 = \sum_{i=0}^{k} i \times 2^i$$

$$S_1 = 0(2^0) + 1(2^1) + 2(2^2) + 3(2^3) + \cdots + k - 1(2^{k-1}) + k(2^k)$$

$\frac{2}{1} = 2, \frac{4}{2} = 2, \frac{8}{4} = 2, \frac{16}{8} = 2, \ldots, \frac{2^k}{2^{k-1}} = 2^{k-(k-1)} = 2^1 = 2,$

$\boldsymbol{common\ ratio}: 2$

$\boldsymbol{Multiplying\ both\ sides\ by\ common\ ratio\ `2`\ ,we\ get}$:

$$2S_1 = 0(2^1) + 1(2^2) + 2(2^3) + \cdots + k - 1(2^k) + k(2^{k+1})$$

$\boldsymbol{We\ can\ rewrite\ it\ as}$:

$$2S_1 = 0(2^0) + 0(2^1) + 1(2^2) + 2(2^3) + \cdots + (k-2)2^{k-1}$$
$$+k - 1(2^k) + k(2^{k+1})$$

$\boldsymbol{Now, subtract\ this\ new\ equation\ (2S_1)\ from\ the\ original\ equation\ (S_1)}$:

$$s_1 - 2S_1 = (0-0)2^0 + (1-0)2^1 + (2-1)2^2 + (3-2)2^3$$
$$+ \cdots + \big(k - 1 - (k-2)\big)2^{k-1} + \big(k - (k-1)\big)2^k$$
$$+(0 - k2^{k+1})$$

$$s_1 - 2S_1 = 0 + 2 + 4 + 8 + \cdots + 2^{k-1} + 2^k - k2^{k+1}$$

$$\Longrightarrow -S_1 = 0 + 2 + 4 + 8 + \cdots + 2^{k-1} + 2^k - k2^{k+1}$$

$(0 + 2 + 4 + 8 + \cdots + 2^{k-1} + 2^k)$ *represents geometric series*: $1 \times 2^1 + 1 \times 2^2 + 1 \times 2^3 + \cdots + 1 \times 2^{k-1} + 1 \times 2^k$

$$\boldsymbol{S(n) = ar^1 + ar^2 + \cdots + ar^n}$$

then this is a geometric series with:

- $\boldsymbol{First\ term: ar}$
- $\boldsymbol{Common\ ratio: r}$

Multiply by $\boldsymbol{r}$:

$$\boldsymbol{rS(n) = ar^2 + ar^3 + \cdots + ar^{n+1}}$$

Now substract:

$$\boldsymbol{S(n) - rS(n) = (ar^1 + ar^2 + \cdots + ar^n) - (ar^2 + ar^3 + \cdots + ar^{n+1})}$$

$$\boldsymbol{= (ar^1 + ar^2 - ar^2 + ar^3 - ar^3 \ldots + ar^n - ar^n + 0 - ar^{n+1})}$$

$$\boldsymbol{= (ar^1 - ar^{n+1})}$$

$$\boldsymbol{S(n) - rS(n) = (ar - ar^{n+1})}$$

$$\boldsymbol{(1 - r)S(n) = (ar - ar^{n+1})}$$

$$\boldsymbol{S(n) = \frac{(ar - ar^{n+1})}{1 - r}}$$

$$\boldsymbol{or, S(n) = \frac{ar^{n+1} - ar}{r - 1}} \left[\boldsymbol{\begin{matrix} Multiplying - 1\ in\ both\ numerator \\ and\ denominator, we\ get \end{matrix}}\right]$$

$$\boldsymbol{or, S(n) = \frac{ar(r^n - 1)}{r - 1}}$$

$$S(n) = \boldsymbol{a}\left(\frac{\boldsymbol{r}(\boldsymbol{r}^{\boldsymbol{n}} - \boldsymbol{1})}{\boldsymbol{r} - \boldsymbol{1}}\right) = \boldsymbol{1}\left(\frac{\boldsymbol{2}(\boldsymbol{2}^{\boldsymbol{k}} - \boldsymbol{1})}{\boldsymbol{2} - \boldsymbol{1}}\right) = \boldsymbol{2}^{\boldsymbol{k+1}} - \boldsymbol{2}.$$

$$\therefore -\boldsymbol{S_1} = \boldsymbol{2}^{\boldsymbol{k+1}} - \boldsymbol{2} - \boldsymbol{k2}^{\boldsymbol{k+1}}$$

$To\ cancel\ out - 1\ , Multipying - 1\ in\ both\ the\ sides$:

$$\therefore \boldsymbol{S_1} = -\boldsymbol{2}^{\boldsymbol{k+1}} + \boldsymbol{2} + \boldsymbol{k2}^{\boldsymbol{k+1}}$$

$$= \boldsymbol{k2}^{\boldsymbol{k+1}} - \boldsymbol{2}^{\boldsymbol{k+1}} + \boldsymbol{2}$$

$$= \boldsymbol{2}^{\boldsymbol{k+1}}(\boldsymbol{k} - \boldsymbol{1}) + \boldsymbol{2}$$

$$\therefore \boldsymbol{S_1} = \boldsymbol{2}^{\boldsymbol{k+1}}(\boldsymbol{k} - \boldsymbol{1}) + \boldsymbol{2}$$

$$S_2 = \sum_{i=0}^{k} 2^i$$

$$= \left(1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2 + 1 \times 2^3 + \cdots + 1 \times 2^{k-1} + 1 \times 2^k\right)$$

Applying geometric progression(series) :

$$S(n) = ar^0 + ar^1 + ar^2 + \cdots + ar^n = a\left(\frac{r^{n+1} - 1}{r - 1}\right), where$$

$S(n)$ is sum of first n terms , a is coefficient and r is common ratio between the consecutive terms.

We get:

$$S_2 = \sum_{i=0}^{k} 2^i = 1\left(\frac{2^{k+1} - 1}{2 - 1}\right) = 2^{k+1} - 1.$$

$$T(N) = c_1(S_1) + c_2(s_2) + (N + 1) \times T(0)$$

$$T(N) = c_1\left(2^{k+1}(k - 1) + 2\right) + c_2\left(2^{k+1} - 1\right) + (N + 1) \times T(0)$$

$Now, k = \log_2(N + 1)$, ***hence :***

$$T(N) = c_1\left(2^{\log_2(N+1)+1}(\log_2(N+1)-1)+2\right) + c_2\left(2^{\log_2(N+1)+1}-1\right) + (N+1)\times T(0)$$

$$Now, 2^{\log_2(N+1)+1} = 2^{\log_2(N+1)} \times 2^1 = 2(N+1)$$

$$= c_1(2(N+1)(\log_2(N+1)-1)+2) + c_2(2(N+1)-1) + (N+1)\times T(0)$$

$$= c_1(2(N+1)(\log_2(N+1)-1)+2) + c_2(2(N+1)-1) + (N+1)\times T(0)$$

$$Lets\ solve:\ 2(N+1)(\log_2(N+1)-1)+2$$

$$= 2(N+1)(\log_2(N+1)) - 2(N+1) + 2$$

$$= 2(N+1)(\log_2(N+1)) - 2N - 2 + 2$$

$$= 2(N+1)(\log_2(N+1)) - 2N$$

$$= 2\left((N+1)(\log_2(N+1)-N\right)$$

$$T(N) = c_1\left(2\left((N+1)(\log_2(N+1)-N\right)\right) + c_2(2N+2-1) + (N+1)\times T(0)$$

$$T(N) = c_1\left(2\left((N+1)(\log_2(N+1)-N\right)\right) + c_2(2N+1)$$

$$+(N+1)\times T(0)$$

$Substituting\ T(0) = 1\ ,we\ get$:

$$\boldsymbol{T(N) = c_1\left(2\left((N+1)(\log_2(N+1) - N\right)\right) + c_2(2N+1) + (N+1) \times 1}$$

$$\boldsymbol{T(N) = 2c_1(N+1)(\log_2(N+1) - 2c_1N + c_2(2N+1) + (N+1)}$$
$$\boldsymbol{T(N) = 2c_1(N+1)(\log_2(N+1) - 2c_1N + 2c_2N + c_2 + N + 1}$$

$$\boldsymbol{T(N) = 2c_1(N+1)(\log_2(N+1) + N(-2c_1 + 2c_2 + 1) + (c_2 + 1)}$$

The dominant terms is:

$$\boldsymbol{2c_1(N+1)(\log_2(N+1)}$$

$$\boldsymbol{Therefore: \Theta\left(2c_1(N+1)(\log_2(N+1))\right)}$$
$$\boldsymbol{Or, \quad \Theta\left((N+1)(\log_2(N+1))\right)}$$

$$\boldsymbol{Since, \Theta(N+1) = \Theta(N)}$$

$$\boldsymbol{, \Theta\left((N+1)(\log_2(N+1))\right) = \Theta(N\log_2 N)}$$

Hence actual time complexity for balance tree , here it will be :

$$\boxed{\boldsymbol{\Theta(N\log_2 N)}}$$

Upper Bound $\mathbf{O}(N \log_2 N)$:

$$T(N) = 2c_1(N+1)(\log_2(N+1) + N(-2c_1 + 2c_2 + 1) + (c_2 + 1)$$

Let $A = 2c_1, B = (-2c_1 + 2c_2 + 1), C = (c_2 + 1)$

$$T(N) = A(N+1)\log_2(N+1) + BN + C.$$

We can say,

$$N \leq N \log_2 N$$

for $N \geq 2$,

$$1 \leq N \leq N \log_2 N$$

Hence,

$$C \leq CN \log_2 N$$

$$T(N) = A(N+1)\log_2(N+1) + BN + C.$$

$$T(N) \leq A(N+1)\log_2(N+1) + BN \log_2 N + CN \log_2 N.$$

Combining $B + C$ ***into a new constant*** D

$$T(N) \leq A(N+1)\log_2(N+1) + DN \log_2 N.$$

$$(N+1)\log_2(N+1) \text{ and } N\log_2 N \text{ grow at same rate}.$$

For $N \geq 2$,

$$N + 1 \leq 2N$$

and

$$\log_2(N+1) \leq \log_2(2N)$$
$$\log_2(N+1) \leq \log_2 2 + \log_2 N$$
$$\log_2(N+1) \leq 1 + \log_2 N$$

And,

$$1 + \log_2 N \leq 2\log_2 N$$

$$\therefore \log_2(N+1) \leq 2\log_2 N$$

$$\Rightarrow (N+1)\log_2(N+1) \leq (2N)(2\log_2 N)$$

$$\Rightarrow (N+1)\log_2(N+1) \leq 4N\log_2 N$$

Subsituting:

$$T(N) \leq A(N+1)\log_2(N+1) + DN\log_2 N$$

$$T(N) \leq A4N \log_2 N + DN \log_2 N$$

$$T(N) \leq (4A + D)N \log_2 N$$

$$Let\ , 4A + D = M$$

$$T(N) \leq M \cdot N \log_2 N$$

$$That's\ the\ upper\ bound\ proving:$$

$$T(N) = \mathbf{O}(N \log_2 N)$$

$$Lower\ Bound\ \Omega(N \log_2 N):$$

$$Look\ at\ the\ dominant\ term:$$

$$A(N + 1) \log_2 (N + 1)$$

$$The\ remaining\ terms\ are\ only\ linear:$$

$$BN + C = \mathbf{O}(N)$$

$$Since$$

$$N = o(N \log_2 N)$$

How?

$$\boldsymbol{BN + C = O(N) grows\ at\ most\ linearly.}$$

Now compare it with $\boldsymbol{N \log_2 N}$:

Consider:

$$= \frac{\boldsymbol{BN + C}}{\boldsymbol{N \log_2 N}}$$

Let, $\boldsymbol{B = 1}$ ***and*** $\boldsymbol{C = 0}$ ***, then***:

$$= \frac{\boldsymbol{N}}{\boldsymbol{N \log_2 N}}$$

$$= \frac{\boldsymbol{1}}{\boldsymbol{\log_2 N}}$$

Applying $\boldsymbol{\lim_{n \to \infty} \frac{1}{\log_2 N}}$ ***, we get***:

$$\boldsymbol{\Longrightarrow \lim_{n \to \infty} \frac{1}{\log_2 N} = \frac{\lim_{n \to \infty} 1}{\lim_{n \to \infty} \log_2 N} = \frac{1}{\infty} = 0}$$

$$\boldsymbol{i.e. when\ n \to \infty, \frac{1}{\log_2 N} \to 0}$$

$$i.e. when\ n \to \infty, \frac{BN + C}{N \log_2 N} \to 0$$

$$hence\ , BN + C = o\left(N \log_2 N\right)$$

$$\therefore T(N) = A(N+1)\log_2(N+1) + o\left(N \log_2 N\right).$$

$$the\ linear\ term : BN + C\ \ becomes\ insignificant\ compared\ with\ N \log_2 N$$

$$So, for\ sufficiently\ large\ N:$$

$$T(N) \geq \frac{A}{2}(N+1)\log_2(N+1).$$

$$There\ fore,$$

$$As\ , N + 1 \geq N$$

$$\Rightarrow\ \log_2(N+1) \geq \log_2(N)$$

$$\Longrightarrow, (N+1)\log_2(N+1) \geq N \log_2(N)$$

$$\Longrightarrow \frac{A}{2}(N+1)\log_2(N+1) \geq \frac{A}{2} N \log_2(N)$$

$$T(N) \geq \frac{A}{2}(N+1)\log_2(N+1) \Longrightarrow T(N) \geq \frac{A}{2} N \log_2 N$$

$$Let\ m = \frac{A}{2}, then:$$

$$T(N) \geq mN \log_2 N$$

That's the lower bound proving:

$$T(N) = \Omega(N \log_2 N)$$

$$Hence\ , mN \log_2 N \leq T(N) \leq M \cdot N \log_2 N$$

$$Or, \Omega(N \log_2 N) \leq \Theta(N \log_2 N) \leq O(N \log_2 N)$$

$\therefore$

$$\boxed{\Theta(N \log_2 N)}$$

Exists.

Now, we have generalized :

$$2T\left(\frac{N-1}{2}\right)+C\,,Base\ Case: T(0)=1$$

$$\Theta(N)$$

$$\neq (NOT\ EQUAL\ TO)$$

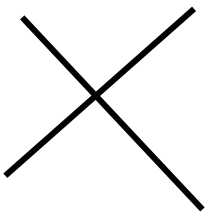
$$2T\left(\frac{N-1}{2}\right)+c_1\times k+c_2\,,Base\ Case: T(0)=1.$$

$$\Theta(N\log_2 N) \qquad [Correct]$$

Which is Actual time complexity for Balance Tree of printlist .

Hence we cannot consider this generalized version :

$$2T\left(\frac{N-1}{2}\right)+C\,,Base\ Case: T(0)=1$$



$$[Wrong]$$

Hence , we will strictly rely on:

$$\mathbf{2}\boldsymbol{T}\left(\frac{\boldsymbol{N}-\mathbf{1}}{\mathbf{2}}\right)+\boldsymbol{c}_{\mathbf{1}}\times\boldsymbol{k}+\boldsymbol{c}_{\mathbf{2}}\text{ , }\boldsymbol{Base\ Case}\text{: }\boldsymbol{T}(\mathbf{0})=\mathbf{1}.$$

$$\boldsymbol{\Theta}(\boldsymbol{N}\log_{\mathbf{2}}\boldsymbol{N})$$
